

M10 / PROTECTED ANSWERS

M10 Explained Reviews

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first	8
Open only after a genuine saved attempt; then close the review and solve the changed retry. . .	8
DE10-01 Why containers exist	8
What a strong answer/evidence set contains	8
Reasoning to repair	8
Most important failure to catch	8
Changed retry	8
Do not call Mastered yet if...	8
DE10-02 Images and containers	8
What a strong answer/evidence set contains	8
Reasoning to repair	9
Most important failure to catch	9
Changed retry	9
Do not call Mastered yet if...	9
DE10-03 Writing a Dockerfile	9
What a strong answer/evidence set contains	9
Reasoning to repair	9
Most important failure to catch	9
Changed retry	9
Do not call Mastered yet if...	10
DE10-04 Ports	10

What a strong answer/evidence set contains	10
Reasoning to repair	10
Most important failure to catch	10
Changed retry	10
Do not call Mastered yet if...	10
DE10-05 Volumes and persistence	10
What a strong answer/evidence set contains	10
Reasoning to repair	10
Most important failure to catch	11
Changed retry	11
Do not call Mastered yet if...	11
DE10-06 Networks and service communication	11
What a strong answer/evidence set contains	11
Reasoning to repair	11
Most important failure to catch	11
Changed retry	11
Do not call Mastered yet if...	11
DE10-07 Compose, configuration and secrets	12
What a strong answer/evidence set contains	12
Reasoning to repair	12
Most important failure to catch	12
Changed retry	12
Do not call Mastered yet if...	12

DE10-08 Why orchestration exists	12
What a strong answer/evidence set contains	12
Reasoning to repair	12
Most important failure to catch	12
Changed retry	13
Do not call Mastered yet if...	13
DE10-09 Airflow architecture and UI	13
What a strong answer/evidence set contains	13
Reasoning to repair	13
Most important failure to catch	13
Changed retry	13
Do not call Mastered yet if...	13
DE10-10 DAGs, tasks and dependencies	13
What a strong answer/evidence set contains	13
Reasoning to repair	14
Most important failure to catch	14
Changed retry	14
Do not call Mastered yet if...	14
DE10-11 TaskFlow, operators and task communication	14
What a strong answer/evidence set contains	14
Reasoning to repair	14
Most important failure to catch	14
Changed retry	14

Do not call Mastered yet if...	15
DE10-12 Scheduling and data intervals	15
What a strong answer/evidence set contains	15
Reasoning to repair	15
Most important failure to catch	15
Changed retry	15
Do not call Mastered yet if...	15
DE10-13 Retries, timeouts and failure behavior	15
What a strong answer/evidence set contains	15
Reasoning to repair	15
Most important failure to catch	16
Changed retry	16
Do not call Mastered yet if...	16
DE10-14 Parameters and task communication	16
What a strong answer/evidence set contains	16
Reasoning to repair	16
Most important failure to catch	16
Changed retry	16
Do not call Mastered yet if...	16
DE10-15 Connections, variables and secrets	17
What a strong answer/evidence set contains	17
Reasoning to repair	17
Most important failure to catch	17

Changed retry	17
Do not call Mastered yet if...	17
DE10-16 Sensors and waiting	17
What a strong answer/evidence set contains	17
Reasoning to repair	17
Most important failure to catch	17
Changed retry	18
Do not call Mastered yet if...	18
DE10-17 Logs and debugging	18
What a strong answer/evidence set contains	18
Reasoning to repair	18
Most important failure to catch	18
Changed retry	18
Do not call Mastered yet if...	18
DE10-18 Catchup, backfills and reprocessing	18
What a strong answer/evidence set contains	18
Reasoning to repair	19
Most important failure to catch	19
Changed retry	19
Do not call Mastered yet if...	19
DE10-19 Integration and production-style orchestration	19
What a strong answer/evidence set contains	19
Reasoning to repair	19

Most important failure to catch 19

Changed retry 19

Do not call Mastered yet if... 20

Read this first

Open this book after saving your own attempt. Compare the reasoning and result, repair the cause of a mistake, then close the answer and try changed inputs.

Open only after a genuine saved attempt; then close the review and solve the changed retry.

DE10-01 | Why containers exist

What a strong answer/evidence set contains

On a learner PC with Docker working, hello-world proves the engine can pull an image and start a container. `docker image ls` shows stored images; `docker ps -a` shows container instances. These commands require a working Docker engine on the learner PC. Save their real output there; static course checks do not substitute for runtime evidence.

Reasoning to repair

Containers improve reproducibility only when the build is reproducible, the configuration boundary is explicit, and persistent state is handled separately. A containerized pipeline can still be non-idempotent, insecure or logically wrong.

Most important failure to catch

A common mistake is saying 'the container includes its own operating system.' Linux containers usually share the host kernel. Another mistake is treating a container as durable storage. If business data exists only in a disposable writable layer, removing the container can remove that state.

Changed retry

A colleague says, 'We use Docker, so dependency pinning no longer matters.' Refute the statement using the difference between an image recipe and an unpinned package install inside that recipe.

Do not call Mastered yet if...

You can explain the review but cannot reproduce the changed task with the review closed, or the lesson requires a live Docker/Airflow runtime event that you have not actually executed. Static and simulator evidence remains valid evidence, but it is not a substitute for a required native runtime.

DE10-02 | Images and containers

What a strong answer/evidence set contains

The image remains after a short-lived container exits. A failed run caused by a deliberately invalid URL does not imply the image vanished. Re-running with corrected runtime configuration creates a new container from the same image.

Reasoning to repair

Promotion usually moves immutable image candidates between environments while environment-specific configuration changes outside the image. This makes rollback and audit much easier than hand-editing running machines.

Most important failure to catch

Do not debug every container problem by rebuilding. A wrong environment variable is a runtime-config problem. Conversely, editing a host source file will not change an already-built image unless that file is bind-mounted or the image is rebuilt.

Changed retry

Imagine two containers created from the same image produce different output because they were given different `SOURCE_URL` values. Explain why this does not violate image immutability.

Do not call Mastered yet if...

You can explain the review but cannot reproduce the changed task with the review closed, or the lesson requires a live Docker/Airflow runtime event that you have not actually executed. Static and simulator evidence remains valid evidence, but it is not a substitute for a required native runtime.

DE10-03 | Writing a Dockerfile

What a strong answer/evidence set contains

The image should build from the documented context and report `appuser` as the runtime user. ``docker history`` should show the build steps. The build is not considered proven in this artifact environment because Docker is absent here; the Dockerfile is statically checked instead.

Reasoning to repair

A professional Dockerfile makes the runtime boundary reviewable: known base, minimal context, deterministic dependency install, explicit work directory, non-root user and a clear foreground process.

Most important failure to catch

Do not `COPY . .` blindly from a large project that contains `.env`, virtual environments or evidence files. Do not put passwords in `ENV` or `RUN` instructions; image history and layers can preserve them. Do not use `root` merely because it avoids permissions learning.

Changed retry

Add a requirements file containing one package. Reorder the Dockerfile so dependency metadata is copied/installed before frequently changing application source. Explain the cache benefit without claiming the cache proves packages are pinned.

Do not call Mastered yet if...

You can explain the review but cannot reproduce the changed task with the review closed, or the lesson requires a live Docker/Airflow runtime event that you have not actually executed. Static and simulator evidence remains valid evidence, but it is not a substitute for a required native runtime.

DE10-04 | Ports

What a strong answer/evidence set contains

`docker compose ps` should show a host mapping similar to `0.0.0.0:8090->8000/tcp`. Changing `SOURCE_PORT` changes only the host entry point; the service-to-service port remains 8000.

Reasoning to repair

Expose only interfaces that need host/external access. Databases and internal services often communicate on private container networks without publishing every port to the host.

Most important failure to catch

`EXPOSE 8000` does not publish port 8000. Publishing `8090:8000` also does not make the source process listen on 8090 inside the container. If a process listens only on `127.0.0.1` inside some container scenarios, publication may still not behave as you expect; always inspect the actual bind/listen configuration.

Changed retry

Port 8090 is already occupied on your laptop. Design the smallest safe correction and explain why rebuilding the image would be unrelated to the conflict.

Do not call Mastered yet if...

You can explain the review but cannot reproduce the changed task with the review closed, or the lesson requires a live Docker/Airflow runtime event that you have not actually executed. Static and simulator evidence remains valid evidence, but it is not a substitute for a required native runtime.

DE10-05 | Volumes and persistence

What a strong answer/evidence set contains

After ordinary down and up, the named volume still exists. After `down -v`, it is removed. The source bind mount remains on the host because it was never Docker-managed volume data.

Reasoning to repair

State ownership is an architectural decision. Volumes survive container replacement but do not by themselves provide backup, replication, retention or corruption recovery.

Most important failure to catch

Never teach `docker compose down -v` as routine cleanup when the volume contains data you care about. Also avoid bind-mounting broad host folders with write access just to bypass permissions. Mount only what the service needs.

Changed retry

A teammate says 'the database is in a container, so backups are unnecessary because the volume persists.' Explain why persistence and backup/recovery are different controls.

Do not call Mastered yet if...

You can explain the review but cannot reproduce the changed task with the review closed, or the lesson requires a live Docker/Airflow runtime event that you have not actually executed. Static and simulator evidence remains valid evidence, but it is not a substitute for a required native runtime.

DE10-06 | Networks and service communication

What a strong answer/evidence set contains

With both services on the same Compose network, the worker can resolve source. The host-facing URL still uses localhost and the published port. The internal URL uses service name plus container port.

Reasoning to repair

Stable service discovery plus health/readiness checks is safer than coding around ephemeral addresses. Network boundaries also help you avoid publishing internal-only services to the host.

Most important failure to catch

Do not fix a service-to-service failure by copying the current container IP into source code. Do not assume `depends_on` means the dependency is ready to serve; start order and readiness are different concepts. A health check or application retry may still be necessary.

Changed retry

Rename the Compose service from source to feed. List every configuration reference that should change and every place that should not need an IP update.

Do not call Mastered yet if...

You can explain the review but cannot reproduce the changed task with the review closed, or the lesson requires a live Docker/Airflow runtime event that you have not actually executed. Static and simulator evidence remains valid evidence, but it is not a substitute for a required native runtime.

DE10-07 | Compose, configuration and secrets

What a strong answer/evidence set contains

`docker compose config` should resolve variables and show service/network/volume structure before any container starts. The worker should eventually write one report based on the served CSV. A clean clone should need only documented environment values, not hidden machine state.

Reasoning to repair

Version-control safe structure and names; inject environment-owned values at runtime; store real secrets in a secrets mechanism appropriate to the deployment environment. Reproducibility means a clean environment can rebuild the same topology from documented inputs.

Most important failure to catch

Never commit real passwords merely because they are in `.env`. `.env` is configuration storage, not automatically a secret vault. Do not use Compose Variables as a substitute for production secret backends. Also do not call down `-v` casually.

Changed retry

Move the host port and output volume name through environment interpolation. Explain why changing those values should not require rebuilding the worker image.

Do not call Mastered yet if...

You can explain the review but cannot reproduce the changed task with the review closed, or the lesson requires a live Docker/Airflow runtime event that you have not actually executed. Static and simulator evidence remains valid evidence, but it is not a substitute for a required native runtime.

DE10-08 | Why orchestration exists

What a strong answer/evidence set contains

Three partitions should produce row counts 4, 3 and 5. Re-running should overwrite the same partition result with the same business totals rather than append duplicate logical output. That behavior is what later makes backfill/retry discussion concrete.

Reasoning to repair

Orchestration adds controlled execution semantics around testable units. A good DAG is a readable dependency model, not a dumping ground for every line of business code.

Most important failure to catch

Do not move all transformation code into a huge DAG file just because Airflow can run Python. Do not make orchestration responsible for fixing bad source semantics. Keep the business work testable outside the scheduler where possible.

Changed retry

The pipeline now has two independent downstream reports after validation. Redraw the graph and explain why the orchestrator can run those two branches concurrently without changing the validation contract.

Do not call Mastered yet if...

You can explain the review but cannot reproduce the changed task with the review closed, or the lesson requires a live Docker/Airflow runtime event that you have not actually executed. Static and simulator evidence remains valid evidence, but it is not a substitute for a required native runtime.

DE10-09 | Airflow architecture and UI

What a strong answer/evidence set contains

Airflow 3.3.1 should be reported on the pinned learner environment, the Dag should appear without import errors, and one test/manual run should expose task/run state in the UI. These are pending native-runtime proofs, not release claims made from this container.

Reasoning to repair

Operational diagnosis improves when you name the layer first: definition/parse, scheduling, task execution, dependency/service, or data correctness. Airflow gives state; your engineering still has to interpret it.

Most important failure to catch

When a Dag is missing, inspect import/parsing errors before reinstalling everything. When a task fails, distinguish Dag parse failure from task runtime failure. UI red does not tell you root cause; logs and task context do.

Changed retry

The scheduler is healthy but a provider import inside one Dag is missing. Explain why restarting the whole machine is a weaker first response than checking Dag import errors and the environment where Airflow parses files.

Do not call Mastered yet if...

You can explain the review but cannot reproduce the changed task with the review closed, or the lesson requires a live Docker/Airflow runtime event that you have not actually executed. Static and simulator evidence remains valid evidence, but it is not a substitute for a required native runtime.

DE10-10 | DAGs, tasks and dependencies

What a strong answer/evidence set contains

The graph is linear for the base lab: extract -> validate -> publish. If validation fails, publish should not run. This is a dependency safety property, not merely a visual preference.

Reasoning to repair

Graph design is operational design: task boundaries determine what can retry independently, what can run concurrently, what evidence is isolated and how failures block downstream side effects.

Most important failure to catch

Do not create fake dependencies to force a pretty left-to-right diagram. Do not split work into tiny tasks that pass huge data through orchestration metadata. Task boundaries should reflect retries, ownership, resources, observability or meaningful dependency boundaries.

Changed retry

A new alert task should run only after validation failure, not as an ordinary downstream success dependency. Describe the trigger/branching requirement rather than connecting it blindly to publish.

Do not call Mastered yet if...

You can explain the review but cannot reproduce the changed task with the review closed, or the lesson requires a live Docker/Airflow runtime event that you have not actually executed. Static and simulator evidence remains valid evidence, but it is not a substitute for a required native runtime.

DE10-11 | TaskFlow, operators and task communication

What a strong answer/evidence set contains

Calling a decorated task while defining the Dag creates a task/XComArg relationship rather than immediately executing the function. In a real run Airflow stores task communication according to its XCom backend; our course keeps messages deliberately small.

Reasoning to repair

Use orchestration metadata for coordination; use data stores for data. This keeps scheduler/metadata systems small, makes retries clearer and avoids coupling workflow correctness to oversized XCom serialization.

Most important failure to catch

Do not confuse Python function call appearance with ordinary eager execution at Dag-definition time. Do not return huge datasets just because it works in a toy local environment. Do not use a global Variable to pass per-run task output.

Changed retry

Replace a returned full list of 100,000 IDs with an object-storage/database path plus count/checksum metadata. Explain how this improves operational robustness.

Do not call Mastered yet if...

You can explain the review but cannot reproduce the changed task with the review closed, or the lesson requires a live Docker/Airflow runtime event that you have not actually executed. Static and simulator evidence remains valid evidence, but it is not a substitute for a required native runtime.

DE10-12 | Scheduling and data intervals

What a strong answer/evidence set contains

Historical runs map to historical partitions deterministically. The Dag uses a timezone-aware start date. The course treats `data_interval_start/end` as the semantic boundary; it does not teach deprecated 'execution date means run-now time' intuition.

Reasoning to repair

Interval-correct workflows make backfills, retries and audits possible. Time semantics are part of data correctness, not a scheduler trivia question.

Most important failure to catch

`date.today()` is wrong for replayable partition selection. An offset-naive start date creates ambiguity. Changing catchup without understanding what historical runs will be created can unexpectedly process or skip partitions.

Changed retry

Change from daily to hourly processing. State what one data interval represents and how the partition path needs more than a date to stay unique.

Do not call Mastered yet if...

You can explain the review but cannot reproduce the changed task with the review closed, or the lesson requires a live Docker/Airflow runtime event that you have not actually executed. Static and simulator evidence remains valid evidence, but it is not a substitute for a required native runtime.

DE10-13 | Retries, timeouts and failure behavior

What a strong answer/evidence set contains

A simulated transient error records one failed attempt and then success. A deterministic invalid-row condition fails validation and blocks publish. Re-running a successful partition does not multiply logical output.

Reasoning to repair

Retry policy is part of side-effect design. Reliable orchestration distinguishes transient infrastructure failures from deterministic data/config defects and leaves enough evidence to diagnose both.

Most important failure to catch

Do not catch every exception and return success. Do not retry authentication errors or schema violations indefinitely. Do not add retries around a non-idempotent write without first defining duplicate/transaction behavior.

Changed retry

A publish task sends an external payment request and you cannot prove idempotency. Decide whether automatic retry is safe and name the external idempotency key/transaction control you would require.

Do not call Mastered yet if...

You can explain the review but cannot reproduce the changed task with the review closed, or the lesson requires a live Docker/Airflow runtime event that you have not actually executed. Static and simulator evidence remains valid evidence, but it is not a substitute for a required native runtime.

DE10-14 | Parameters and task communication

What a strong answer/evidence set contains

A manual trigger with an invalid Param should be rejected by Airflow's validation instead of reaching task code. A scheduled run uses the default. The value is small configuration, not a substitute for sending datasets between tasks.

Reasoning to repair

Parameters make controlled variation explicit. Keep them small, typed and business-readable; keep observed run facts in task outputs/metadata and bulk data in data stores.

Most important failure to catch

Do not read manually supplied Param values from `dag.params` inside top-level definition code and assume you have run-time overrides; Airflow documentation distinguishes defaults from `DagRun/task` context. Do not store per-run task outputs in global Variables.

Changed retry

A teammate wants a Param containing 50,000 customer IDs. Redesign the interface so the Param contains a durable input reference plus validation metadata instead.

Do not call Mastered yet if...

You can explain the review but cannot reproduce the changed task with the review closed, or the lesson requires a live Docker/Airflow runtime event that you have not actually executed. Static and simulator evidence remains valid evidence, but it is not a substitute for a required native runtime.

DE10-15 | Connections, variables and secrets

What a strong answer/evidence set contains

A clean repository contains no real password/token literals. Moving between environments should replace connection/secret configuration without editing business DAG logic. Global Variables are not used as a database for per-run task data.

Reasoning to repair

Portable orchestration keeps code stable while environment-owned connection/secrets configuration changes behind stable identifiers. Least privilege and log sanitization remain required.

Most important failure to catch

If a secret was committed, moving it to a Variable after the fact does not erase Git history; rotate it and follow repository history-removal policy. Do not print connection URIs/tokens into task logs. Do not confuse environment interpolation with secret protection.

Changed retry

Your company moves from Airflow DB-stored Connections to Vault. Explain why well-designed DAG code using the same connection ID may not need business-logic changes.

Do not call Mastered yet if...

You can explain the review but cannot reproduce the changed task with the review closed, or the lesson requires a live Docker/Airflow runtime event that you have not actually executed. Static and simulator evidence remains valid evidence, but it is not a substitute for a required native runtime.

DE10-16 | Sensors and waiting

What a strong answer/evidence set contains

A sensor succeeds only when its condition is true; it should fail after the declared timeout rather than wait forever. Deferrable support depends on the provider/runtime and must be proven in the learner environment.

Reasoning to repair

Waiting is capacity planning. A correct sensor defines condition, cadence, timeout, resource mode and failure ownership; otherwise a quiet wait can be as operationally expensive as a failed task.

Most important failure to catch

Do not write `while True: sleep(...)` inside an ordinary worker task as your default waiting strategy. Do not use a 1-second poke interval for a file expected once per hour. Do not omit timeout and create endless invisible waiting.

Changed retry

The external platform can emit an event instead of being polled. Explain why event-driven/deferred waiting may be better than frequent polling and what failure/timeout behavior still needs definition.

Do not call Mastered yet if...

You can explain the review but cannot reproduce the changed task with the review closed, or the lesson requires a live Docker/Airflow runtime event that you have not actually executed. Static and simulator evidence remains valid evidence, but it is not a substitute for a required native runtime.

DE10-17 | Logs and debugging

What a strong answer/evidence set contains

A failure can be traced to a specific partition/attempt and compared with a later repaired attempt. The logs contain enough context to reproduce the problem but no password/token literals.

Reasoning to repair

Operationally useful logging connects workflow state to data evidence. Preserve failed attempts, sanitize sensitive values, and design logs so another engineer can diagnose without reproducing your entire local setup.

Most important failure to catch

Do not log entire dataframes or response bodies containing PII/secrets merely because debugging is hard. Do not erase failed logs/evidence after repair. Do not read only the final exception line when the earlier stack/context shows the actual cause.

Changed retry

A task is green but downstream total is wrong. Explain why the debugging path should move from orchestration status to data reconciliation/logic evidence rather than adding more retries.

Do not call Mastered yet if...

You can explain the review but cannot reproduce the changed task with the review closed, or the lesson requires a live Docker/Airflow runtime event that you have not actually executed. Static and simulator evidence remains valid evidence, but it is not a substitute for a required native runtime.

DE10-18 | Catchup, backfills and reprocessing

What a strong answer/evidence set contains

Each date has one deterministic output file. Reprocessing Sep 2 replaces/converges that partition rather than appending a second logical copy. A historical range can be reasoned about before using Airflow's live backfill commands.

Reasoning to repair

Historical recovery is a first-class reliability feature. Partition identity, side-effect idempotency and explicit range controls determine whether backfill restores trust or creates duplicate damage.

Most important failure to catch

Do not backfill a non-idempotent external write without duplicate/transaction controls. Do not let historical runs overwrite a shared `latest.csv`. Do not assume `catchup=True` is always desirable; choose based on whether missed historical intervals should run.

Changed retry

A downstream API cannot safely accept the same event twice. Introduce an idempotency key or staging/upsert boundary and explain how that changes backfill safety.

Do not call Mastered yet if...

You can explain the review but cannot reproduce the changed task with the review closed, or the lesson requires a live Docker/Airflow runtime event that you have not actually executed. Static and simulator evidence remains valid evidence, but it is not a substitute for a required native runtime.

DE10-19 | Integration and production-style orchestration

What a strong answer/evidence set contains

Static/local QA proves the files are internally coherent and the simulator demonstrates deterministic partition behavior. It does NOT prove Docker Engine, Airflow scheduler/UI or DagRun behavior on the learner PC. Those remain explicit runtime checks before calling the module fully native-validated.

Reasoning to repair

M10 mastery means you can defend both layers: environment reproducibility and workflow reliability. The G04 gate tests fresh pipeline reasoning after this module; it is not passed by rereading the final lesson.

Most important failure to catch

Do not use a passing static parser as evidence that Docker networking works. Do not use a green Airflow task as evidence that totals are correct. Do not hide environment blockers; record them and continue with the parts that can be truthfully validated.

Changed retry

The business switches from daily CSV to hourly object-store files. Identify which Docker pieces can remain unchanged, which Dag schedule/partition logic changes, and which validation/backfill controls must be retested.

Do not call Mastered yet if...

You can explain the review but cannot reproduce the changed task with the review closed, or the lesson requires a live Docker/Airflow runtime event that you have not actually executed. Static and simulator evidence remains valid evidence, but it is not a substitute for a required native runtime.